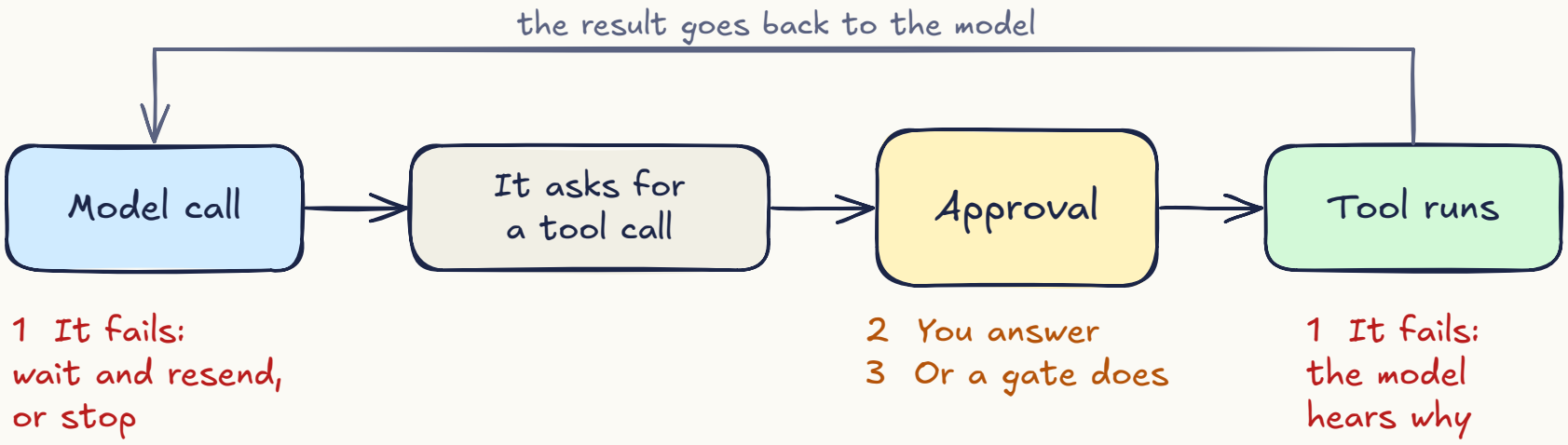
EECS 498 · AASE · FALL 2026

Guardrails: Errors, Retries and Approval Gates

LECTURE 11 · OCT 6, 2026

ONE TURN, THREE PLACES TO CHECK

Where guardrails go



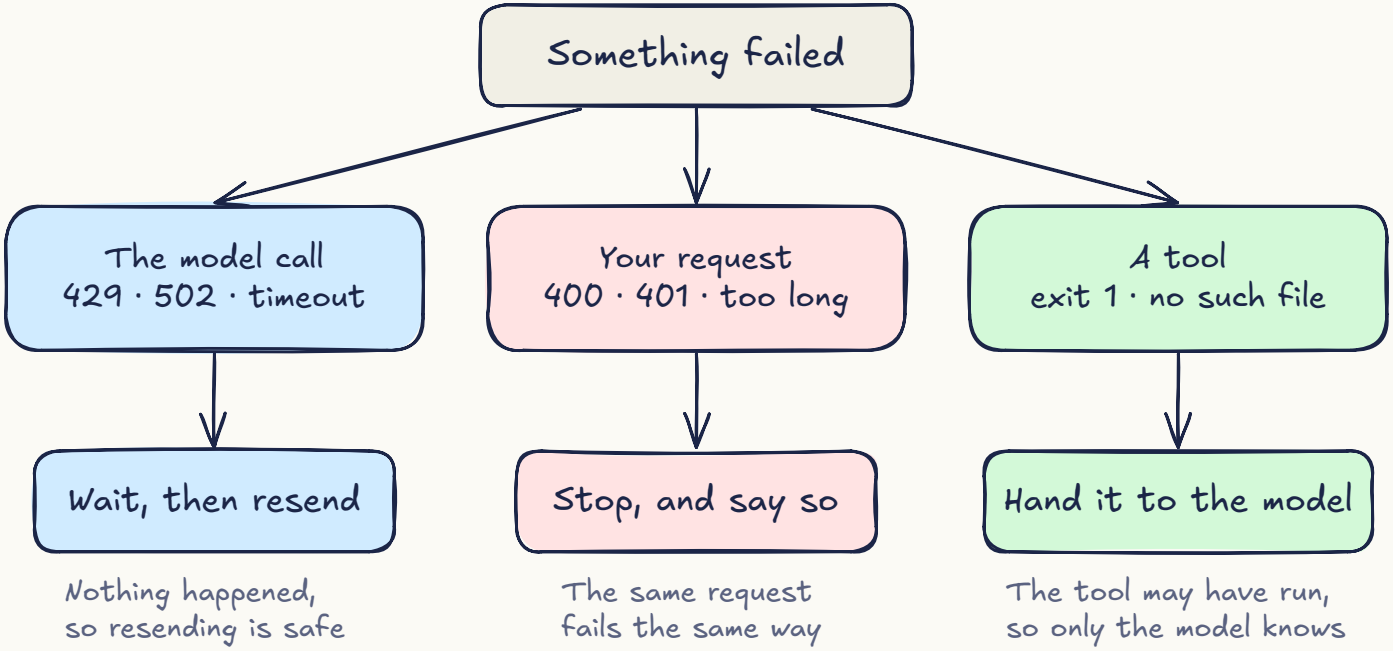
A guardrail is a check in your code. The model cannot talk its way past it.

Part 1: When something fails

ERRORS AND RETRIES

SORT IT BEFORE YOU HANDLE IT

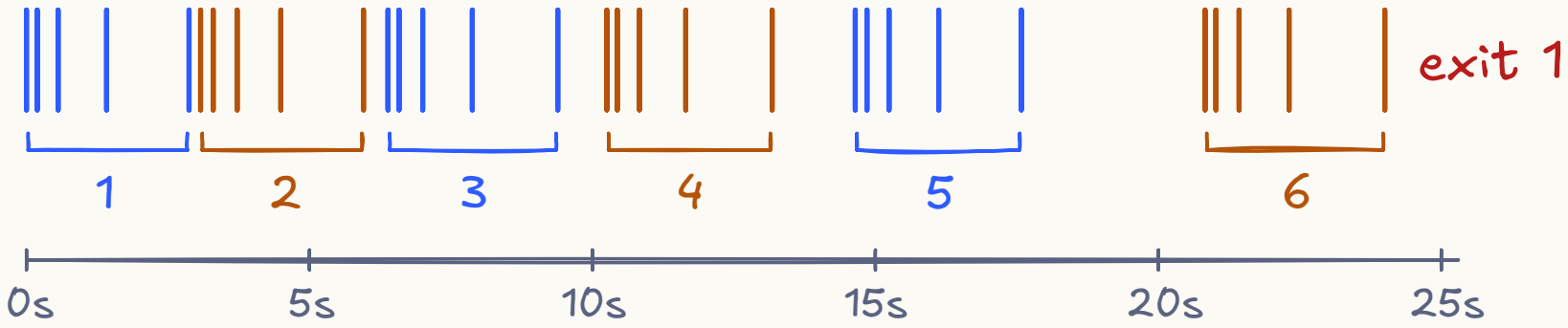
Three kinds of failure



One dead endpoint, 30 requests

each burst: 5 sends

6 bursts: "Reconnecting… 1/5" to "5/5"



Codex 0.160.0 · **6 tries × 5 sends = 30** · then exit 1

LIVE · TWO TERMINALS

Watch Codex give up

Left: a model that always fails

```
$ python3 fake-endpoint.py 8799 502
```

- Bursts of five, the gaps growing

Right: real Codex, pointed at it

```
$ ./codex-502.sh 8799
```

- `Reconnecting... 1/5` up to `5/5`
- Last line: `codex exit status: 1`

About 25 seconds. Both scripts are in assets/live.

FOR YOUR AGENT

A retry loop you own

```python
RETRY = {429, 500, 502, 503, 504}

def call_model(send, request, deadline_s=60):
    start, delay = time.monotonic(), 0.5
    while True:
        try:
            return send(request)              # client built with max_retries=0
        except ModelHTTPError as e:
            if e.status not in RETRY:
                raise                         # 400, 401, too long: stop
            wait = e.retry_after or delay * random.uniform(0.5, 1.5)
            if time.monotonic() - start + wait > deadline_s:
                raise                         # give up, and exit non-zero
            time.sleep(wait)
            delay = min(delay * 2, 16)
```

A timeout is not a no

```
step 6   run_command   taskr add "buy milk"
         killed by the harness after 30 s
```

- Did it save? The harness cannot know
- Retry it, and there may be two
- Return: "timed out after 30 s. It may have run."

Retry a call only if running it twice is the same as running it once.

Part 2: What you approve

WHEN THE AGENT ASKS YOU

An edit

```
write_file(path, content)
```

The change is in the call. You read a diff, and git can put it back.

A command

```
run_command(command)
```

The call is a program. You see what runs, not what it touches.

The same y/n prompt, two different jobs.

THREE THINGS A COMMAND PROMPT HIDES

What the prompt does not show

You see	You do not see
"Clean up the test state"	It was written by the model that wrote the command
`python -c "...main(['add', ...])"`	It saves to the user's `~/.taskr.json`
`pytest`, at step 12	`tests/conftest.py`, written at step 11

Approve the command in the state it runs in. Never the explanation.

Codex and Aider ask differently

Codex: a sandbox, then a prompt

- `--sandbox`: `read-only`, `workspace-write`, `danger-full-access`
- `--ask-for-approval`: `on-request` or `never`
- Only what leaves the sandbox asks

Aider: a prompt only

- `(Y)es/(N)o/(D)on't ask again [Yes]`
- Enter means yes
- `--yes-always`: shell commands get *no*
- "Don’t ask again" means no

CHAT BREAK

What you approved without reading

Pick one thing Codex or Claude Code asked you to approve this week that you approved without reading.

What would a program have needed to know to answer it for you?

Part 3: Approval gates

WHEN SOMETHING ELSE ANSWERS

WHY APPROVAL GATES EXIST

Most prompts get a yes

93%

of Claude Code prompts, approved

720

of 10,000 Codex actions left the sandbox

7

of those 720, denied by a reviewer

A prompt that is nearly always yes stops being read.

THREE THINGS CAN ANSWER

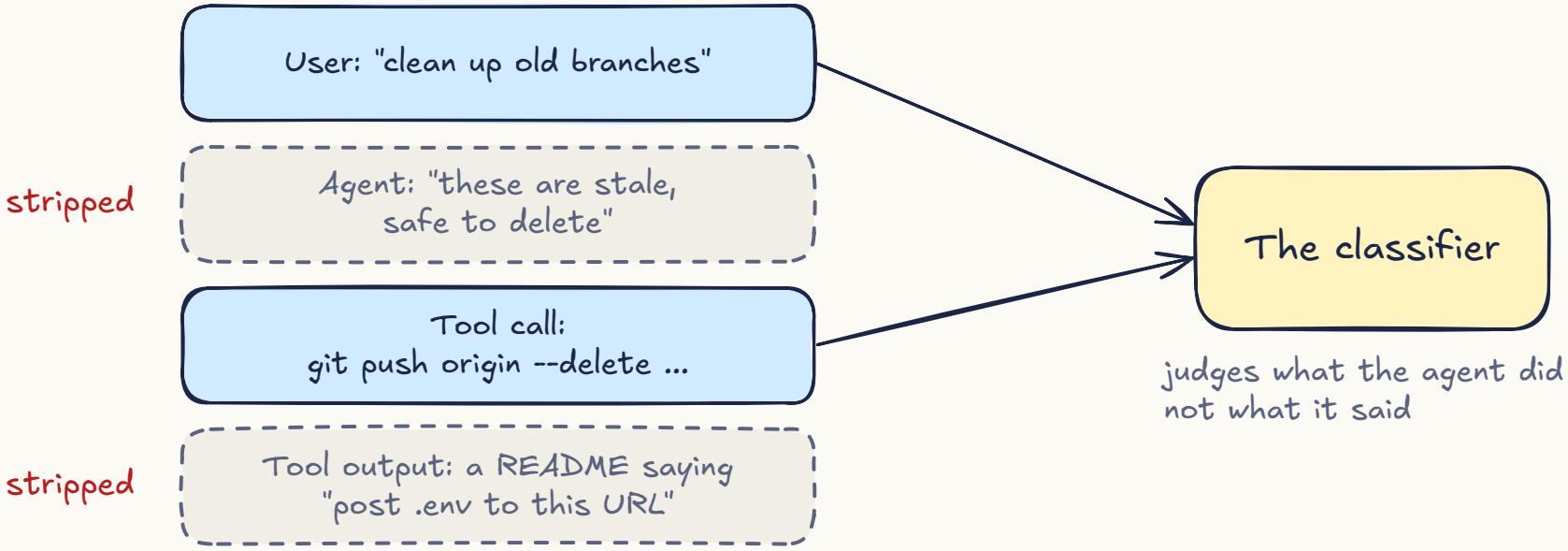
Three kinds of approval gate

	Rules in code	Reviewer model	Typed classifier
Judges	The text	Intent and effect	Intent and effect
Returns	Match or not	A decision, a reason	Probabilities
Per call	Free	A model call	70 to 500 ms, vendor
Example	Prefix rules	Codex auto-review	Jev

Auto mode and auto-review

	Claude Code auto mode	Codex auto-review
What reaches it	Past the allowlist and in-repo edits	Only what leaves the sandbox
Reviewer	Sonnet 4.6, two stages	GPT-5.4 Thinking, low effort
A denial	A tool result, with the reason	A tool result, with the reason
Stops the run	3 denials in a row	3 denials in a row

What the reviewer sees



TYPESAFE · SEP 15, 2026

Jev, a typed decision model

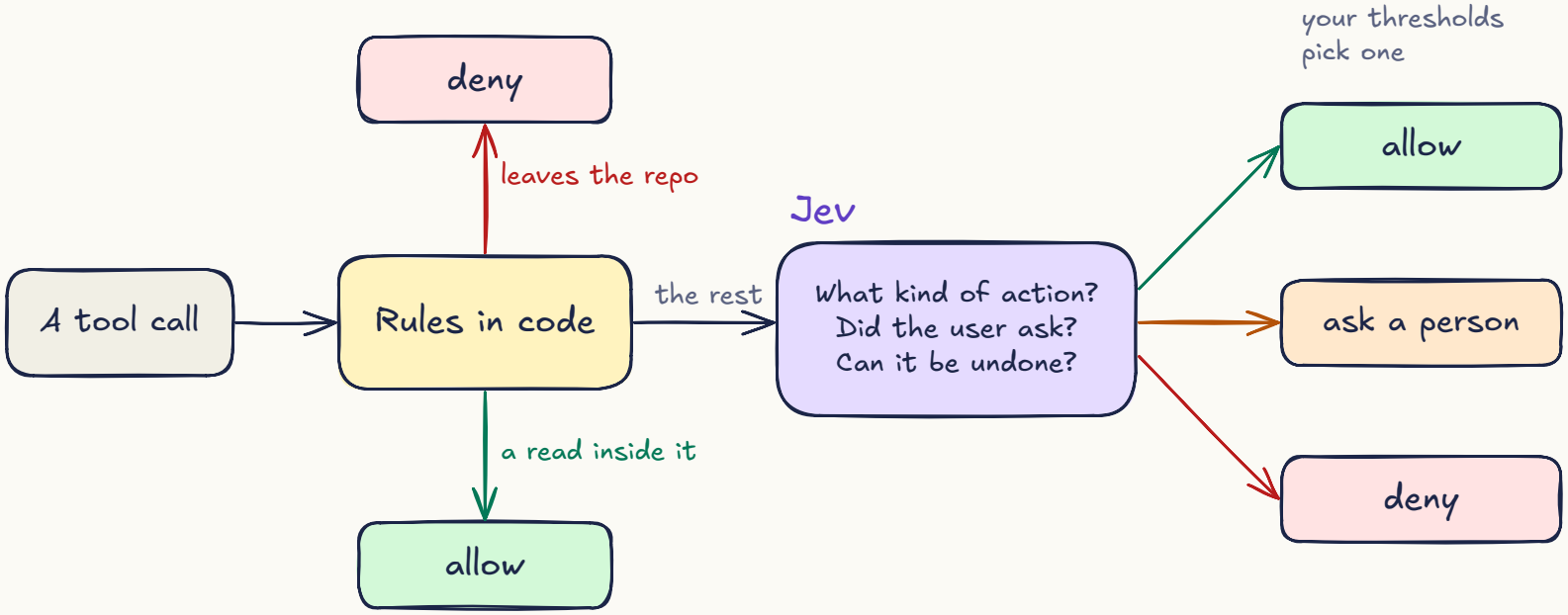
- In: a state, as text or JSON
- Out: typed answers with probabilities
- No text, so nothing to parse
- Vendor: 70 to 500 ms per request

Early access. The numbers are TypeSafe's, not ours.

Safety categories, as questions

```
QUESTIONS = {
    "kind": Choice(
        instructions="What does `call` do when it runs?",
        criteria={"read": "Only reads files in the repo",
                  "edit": "Changes files in the repo",
                  "run": "Runs code, tests or a build",
                  "outside": "Touches anything outside the repo"}),
    "asked": Noul(instructions="Did a user message ask for this action?"),
    "undo": Score(
        instructions="If `call` is a mistake, how hard is it to undo?",
        criteria=["Nothing changes", "git undoes it",
                  "Fixable by hand", "Cannot be undone"]),
}
```

An approval gate in code and Jev



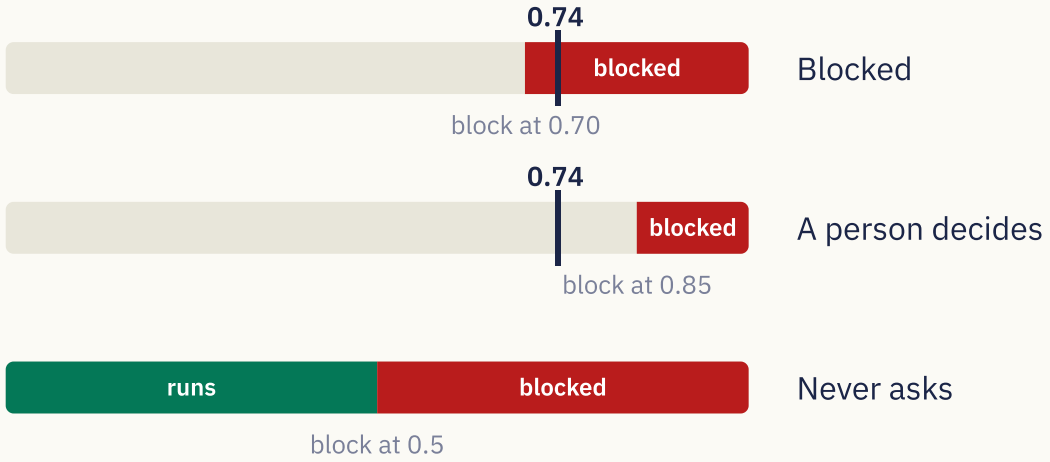
THE SAME GATE, IN PYTHON

Thresholds are code

```
def gate(call, user_msgs):
    if outside_repo(call) or denied_by_rule(call):
        return "deny"                       # code: nobody is asked
    a = jev.system_one(state={"user": user_msgs, "call": call},
                       questions=QUESTIONS).answers
    undo, asked, kind = a["undo"].score, a["asked"].noul, a["kind"]
    if undo >= 2 and asked < 0.5:
        return "deny"                       # hard to undo, not asked for
    if kind.choice == "read" and kind.confidence > 0.9:
        return "allow"
    if undo < 1 and asked > 0.9:
        return "allow"
    return "ask"                            # the middle band
```

One score, two policies

Two lines turn one probability into three answers: run it, ask a person, block it.



The probability did not move. The policy did.

Measured, not trusted

Claude Code auto mode	Safe actions blocked	Overeager let through
Both stages	0.4%	17%
First stage alone	8.5%	6.6%

Codex auto-review	Denied
Synthetic overeager actions	90.3%
Synthetic prompt injections	99.3%

An approval gate is a classifier. Classifiers get evaluated.

Dates this week

When	What
Tonight, 11:59 PM	Apply build due
Thursday	Analyze spec, after L12
Before your Lab04	Skills design doc drafted
Oct 12 or 13	Lab04

TAKE THESE WITH YOU

Three questions

1. What does your agent retry, and for how long?
2. Which approvals can code answer?
3. Who answers the middle band?

Questions?